

Doc. no. P0178R0
Date: 2016-02-15
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Allocators and swap

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
2. [Introduction](#)
3. [The Basic Problem](#)
 - [Feedback from field experience](#)
4. [Proposed Solution](#)
5. [Acknowledgements](#)
6. [References](#)

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Introduction

The C++11 standard introduced `allocator_traits` as a way to easily support a variety of allocator models, especially those where allocators hold some kind of state that might affect allocator, such as a pointer to a memory resource, [N3916](#). One of the problems that had to be tackled was the effect of calling `swap` on containers (or other allocator-aware types) that have allocators that do not compare equal. The committee at the time took the conservative option of simply making this undefined behavior, and a couple of issues have since been filed regarding some of the problems this causes: [LWG #2152](#), [LWG #2153](#). The last time the Library Working Group looked at these issues, it called for a paper to better describe the issues and propose a solution (with wording!) This is that paper.

The Basic Problem

When support for stateful allocators was added to C++11, it became important to answer the question of what should happen when two containers with allocators that cannot allocate/deallocate memory on behalf of the other are swapped. The conservative position adopted at the time was to simply declare such calls out of contract, and make them undefined behavior. This gave us the maximum freedom to revise our position later, if necessary. It also matched the contract Bloomberg documented for their library, which was one of the code bases known to make widespread use of stateful allocators.

The problem with giving the free-function `swap` a *narrow* contract is that it does not satisfy the contract of the generic `swap` function in the same namespace. This greatly harms the ability to use `std::swap` in generic code, as the calling template must now be aware if the type it is instantiated for is a standard library container, and if so, whether its allocators compare equal. Conversely, this is not a problem for the `swap` member function, as the caller either knows exactly which container they are dealing with, or should document the container concept they are dealing with if it differs in some way from the standard. In some sense, the generic free function is more fundamental than the member function.

There has been some concern raised that the standard library cannot make arbitrary constraints on `swap` function found via ADL, so this exercise is vain. That is not entirely true, as if we assume the container requirements apply to user code, then we are already changing their `swap` contract by introducing undefined behavior. However, in practice, the current Container Requirements tables do not document Concepts, but are rather a form of common documentation to avoid redundant wording through each of the container sub-clauses. In this context, the standard library has broken its own `swap` function by adding an overload in its own namespace that, if not present, would allow the container to "do the right thing", and with move-semantics enabled for each container, do so efficiently with the correct exception-safety guarantees. The optimization offered by the overload, in a modern compiler, is tiny (but still worth taking, especially for containers with additional predicates).

The breakage of `swap` has more subtle implications too. The author of generic code could, by defining their own `is_standard_container` trait, rewrite their algorithm to allow for exchanging states of containers with unequal allocators. In fact, this is necessary for any generic algorithm that expects to work with containers unless the contract explicitly calls out this problem. The standard library offers no support for users trying to help their users in this manner. However, the further subtlety is that `is_standard_container` is not sufficient, as `swap` for `pair` and `tuple` is similarly undefined, and would need to be detected and supported separately. This problem further eats into any other type that wraps a standard container and can support stateful allocators.

Feedback from field experience

Bloomberg have over a decade of experience with a stateful allocator model in their standard library. When this was first introduced, the undefined behavior described by the current standard was added to all of our `swap` contracts, but the (undocumented) implementation performs the double-copy/swap described in this paper. This allowed existing code, written without stateful allocators in mind, to continue to function while the new contract was slowly adopted. A decade on, we still get large resistance any time we try to enable assertions on these narrow contracts, as perfectly sensible use-cases exist (frequently, not always, in generic code) for continuing to support this behavior. Given the choice between linear and potentially-throwing `swap` vs. undefined behavior, our users have been very clear which they prefer - as long as the constant-time, non-throwing behavior remains when the allocators do compare equal.

Proposed Solution

The simplest implementation of the `swap` overload (for a given `CONTAINER_TYPE`) should be efficient and correct, without preconditions:

```
void swap(CONTAINER_TYPE & left, CONTAINER_TYPE & right) {
    std::swap<TYPE>(left, right);
}
```

However, the optimal algorithm for swapping containers (or any other allocator-aware type) might be more like:

```
void swap(CONTAINER_TYPE & left, CONTAINER_TYPE & right) {
    if (allocators are compatible) {
        left.swap(right);
    }
    else if (allocator propagation traits are sane) {
        std::swap<TYPE>(left, right);
    }
    else {
        CONTAINER_TYPE tempLeft {std::move(right), left.get_allocator() };
        CONTAINER_TYPE tempRight{std::move(left ), right.get_allocator()};
        swap(left,  tempLeft );
        swap(right, tempRight);
    }
}
```

We assume that containers provide a `swap` member function, which has the precondition that allocators must compare equal (otherwise behavior is undefined, as today). The check that *allocators are compatible* is that they propagate on swap (a compile-time check), or if they are always equal (a compile-time check added as part of C++17), or if they compare equal at run-time. The check that *allocator propagation traits are sane* is a compile-time test which really means that they are the same; either both derive from `true_type`, or both derive from `false_type`. See P0177r0 for more details on this problem, and a proposed resolution that would simplify what we need to consider in this paper (without changing the conclusion).

The final branch is clearly expensive, making two copies of each container, but has the benefit of the strong exception safety guarantee. This is a nice bonus, although not something guaranteed by `std::swap`, and something we prefer to not pay for if we do not need it. However, if we consider the middle branch, what are we really saving, when the allocators do not compare equal? Here is a typical implementation of `std::swap`, with comments of anticipated costs when allocators are different.

```
template <typename TYPE>
void swap(TYPE & left, TYPE & right) {
    TYPE temp{std::move(right)}; // Efficient move construction
    right = std::move(left);      // move-assign-with-different-allocator, copy-swap implementation
    left  = std::move(temp);      // move-assign-with-different-allocator, copy-swap implementation
}
```

Observant readers will want to consider the case that allocators propagate on move-assignment, so that the two assignments in this implementation would be efficient. However, to get to this middle option, we have already considered the case that allocators are *compatible*, which includes the case that they propagate. Hence, we do not get here if allocators propagate for both move-assignment and swap, nor do we get here if they differ due to the sanity check. Likewise, if the container allocators compared equal, we would also have taken the first branch, so we end up in the middle branch only in the case that the

allocators are different, and never propagate. In such cases, the logic of the third and final branch is actually a slight unrolling of what would otherwise take place, while re-ordering work so that the strong exception safety guarantee holds (for free) so the proposed solution is to drop that middle branch.

One final note on exception-safety guarantees are that we do not quite get the strong guarantee in all cases. Associative and unordered associative containers store function objects that are permitted to have throwing swap and assignment operators. The proposed resolution is no worse in such cases, but offers only the basic exception-safety guarantee. The LEWG may separately want to consider whether such support is still warranted as a separate issue, paying particular attention to [LWG #2189](#).

How does this proposal compare to the existing contract? If the allocator propagate on swap, or always compare equal (such as `std::allocator`) then this has exactly the same effect, only we define the free-function `swap` in terms of the member-function, rather than the other way around. If the allocator is stateful, and compares equal at runtime, we have mandated one extra runtime branch to check that allocators compare equal before yielding either the exact same algorithm as before. However, if the allocators do not compare equal, we turn undefined (library) behavior into well-defined behavior. Given the presence of the runtime-check is not observable behavior, this implementation would be perfectly conforming for C++11, or even C++98. Note that this does not mean that existing implementations would not have to change, merely that portable code could not be relying on any of the differences that are exposed.

Also, see paper [P0177R0](#) for a possible direction to simplify further, by requiring all allocators to have *sane* propagation traits.

Proposed Wording

Make the following edits to clause 23 [containers]:

23.2.1 General container requirements [container.requirements.general]

Table 95 “ Container requirements

Expression	Return type	Opertional semantics	Assertion/note pre-/post-condition	Complexity
a.swap(b)	void		<i>Requires:</i> <code>a.get_allocator() == b.get_allocator() allocator_traits<allocator_type>::propagate_on_container_swap::value.</code> <i>post:</i> Exchanges the contents of a and b.	(Note A)
swap(a,b)	void	a.swap(b)	<i>post:</i> e Exchanges the contents of a and b.	(Note A)

9 The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than array, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Lvalues of any Compare, Pred, or Hash types belonging to `a` and `b` shall be swappable and shall be exchanged by calling `swap` as described in 17.6.3.2. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 17.6.3.2. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the ~~swap~~`swap` will have value `b.end()` after the ~~swap~~`swap`. **No exceptions shall be thrown, unless thrown by the swap called for a Compare, Pred, or Hash object.**

10 The expression `swap(a,b)`, for containers `a` and `b` of a standard allocator-aware container type, shall exchange the values of `a` and `b`. Lvalues of any Compare, Pred, or Hash types belonging to `a` and `b` shall be swappable and shall be exchanged by calling `swap` as described in 17.6.3.2. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 17.6.3.2; otherwise, the allocators shall not be swapped and, unless `a.get_allocator() == b.get_allocator()`, iterators into `a` and `b` are invalidated. **No exceptions shall be thrown unless `a.get_allocator() != b.get_allocator() && !allocator_traits<allocator_type>::propagate_on_container_swap::value`, or unless thrown by the swap called for a Compare, Pred, or Hash object. Unless iterators were invalidated, every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap, no move, copy, or swap operations on the individual container elements shall be invoked, and the whole swap operation will have constant complexity; otherwise the swap operation has linear complexity. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.**

1011 If the iterator type of a container belongs to the bidirectional or random access iterator categories (24.2), the container is called *reversible* and satisfies the additional requirements in Table 96.

1112 Unless otherwise specified (see 23.2.4.1, 23.2.5.1, 23.3.3.4, and 23.3.6.5) all container types defined in this Clause meet the following additional requirements:

- 1. - if an exception is thrown by an insert() or emplace() function while inserting a single element, that function has no effects.
- 2. - if an exception is thrown by a push_back(), push_front(), emplace_back(), or emplace_front() function, that function has no effects.
- 3. - no erase(), clear(), pop_back() or pop_front() function throws an exception.
- 4. - no copy constructor or assignment operator of a returned iterator throws an exception.
- 5. - no swap() function throws an exception.
- 6. - no swap() function invalidates any references, pointers, or iterators referring to the elements of the containers being swapped. [Note: The end() iterator does not refer to any element, so it may be invalidated. - end note]

Table 98 Allocator-aware container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
a.swap(b)	void	exchanges the contents of a and b.	Constant

23.2.4.1 Exception safety guarantees [associative.reqmts.except]

3 For associative containers, no swap function throws an exception unless that exception is thrown by the swap of the container's Compare object (if any).

23.2.5.1 Exception safety guarantees [unord.req.except]

3 For unordered associative containers, no swap function throws an exception unless that exception is thrown by the swap of the container's Hash or Pred object (if any).

Note that only sequence containers lack Compare, Pred, or Hash types, so only sequence containers have a requirement to not throw. This constrains only the member function, which retain the no-throw guarantee on swap for all sequence containers in this paper - as member-swap with unequal non-propagating allocators remains undefined behavior.

23.3.2.7 array::swap [array.swap]

3 [Note: Unlike the swap function for other sequence containers, array::swap takes linear time, may exit via an exception, and does not cause iterators to become associated with the other container. - end note]

For the remaining containers, the namespace-scope swap function is completely specified by the container requirements, and does not need to be restated. It is simpler to delete these clauses rather than risk contradictory wording to the general requirements.

23.3.3.5 deque specialized algorithms [deque.special]

```
template <class T, class Allocator>
void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.3.4.7 forward_list specialized algorithms [forwardlist.spec]

```
template <class T, class Allocator>
void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.3.5.6 list specialized algorithms [list.special]

```
template <class T, class Allocator>
void swap(list<T, Allocator>& x, list<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.3.6.6 vector specialized algorithms [vector.special]

```
template <class T, class Allocator>
void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.4.4.5 map specialized algorithms [map.special]

```
template <class Key, class T, class Compare, class Allocator>
void swap(map<Key, T, Compare, Allocator>& x,
          map<Key, T, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.4.5.4 multimap specialized algorithms [multimap.special]

```
template <class Key, class T, class Compare, class Allocator>
void swap(multimap<Key, T, Compare, Allocator>& x,
          multimap<Key, T, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.4.6.3 set specialized algorithms [set.special]

```
template <class Key, class Compare, class Allocator>
void swap(set<Key, Compare, Allocator>& x,
          set<Key, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.4.7.3 multiset specialized algorithms [multiset.special]

```
template <class Key, class Compare, class Allocator>
void swap(multiset<Key, Compare, Allocator>& x,
          multiset<Key, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.5.4.5 unordered_map swap [unord.map.swap]

```
template <class Key, class T, class Hash, class Pred, class Allocator>
void swap(unordered_map<Key, T, Hash, Pred, Allocator>& x,
          unordered_map<Key, T, Hash, Pred, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.5.5.4 unordered_multimap swap [unord.multimap.swap]

```
template <class Key, class T, class Hash, class Pred, class Allocator>
void swap(unordered_multimap<Key, T, Hash, Pred, Allocator>& x,
          unordered_multimap<Key, T, Hash, Pred, Allocator>& y)
```

```
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.5.6.3 unordered_set swap [unord.set.swap]

```
template <class Key, class Hash, class Pred, class Allocator>
void swap(unordered_set<Key, Hash, Pred, Allocator>& x,
          unordered_set<Key, Hash, Pred, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

23.5.7.3 unordered_multiset swap [unord.multiset.swap]

```
template <class Key, class Hash, class Pred, class Allocator>
void swap(unordered_multiset<Key, Hash, Pred, Allocator>& x,
          unordered_multiset<Key, Hash, Pred, Allocator>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(y);
```

The container adapters are not covered under the general container requirements clause, so we do need to revise the wording to use an ADL-lookup swap function.

23.6.3.5 queue specialized algorithms [queue.special]

```
template <class T, class Container>
void swap(queue<T, Container>& x, queue<T, Container>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(x.c, y.c);
```

23.6.4.4 priority_queue specialized algorithms [priqueue.special]

```
template <class T, class Container, class Compare>
void swap(priority_queue<T, Container, Compare>& x,
          priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(x.c, y.c);
swap(x.comp, y.comp);
```

23.6.5.6 stack specialized algorithms [stack.special]

```
template <class T, class Container>
void swap(stack<T, Container>& x, stack<T, Container>& y)
noexcept(noexcept(x.swap(y)));
```

Effects:

```
x.swap(x.c, y.c);
```

Acknowledgements

References

- [LWG #2151](#) basic_string<>::swap semantics ignore allocators
- [LWG #2152](#) Instances of standard container types are not swappable
- [LWG #2153](#) Narrowing of the non-member swap contract
- [N4511](#) Adding [nothrow-]swappable traits, Daniel KrÃ¼gler